

ROI Platform's Jira Style Guide

A basic guide to boringly easy board management

Estimation Philosophy

Before talking about estimations, we have to ground ourselves in what we are most concerned with: [appetites](#).

An appetite is an answer to the question "What are we *willing to spend* to solve this problem?" Notice that the appetite is anchored to the problem, *not* the solution.

Once we know what we're willing to spend, we can begin estimating investment costs for various solutions available to us.

Point System

-  **Base unit:** 1 point = ~1 productive engineering day (4-6h focused work)
-  **Reality check:** 1 point is typically >1 calendar day. I.e. 1 point could have a 2-3 calendar day lead time after accounting for ceremonies, context switching, life, interrupts, etc.
-  **Maximum size:** 4 points (effectively one week of investment)
-  **No 5s:** If it's bigger than 4, break it down

Key Principles

1.  **Aim small, miss small** – Prefer multiple small tickets over large ones
2.  **Monitoring investments, not people** – We're sizing investments to contrast them with our appetite to invest in solving a problem, not predicting exact hours or doing Big Brother Board Management™
3.  **Break down 4s** – Any 4-point ticket should have clear sub-stories or tasks
4.  **Consistency over precision** – [Tight groupings are better than bullseyes](#).
Consistency in our estimates is far more important than precision.

Ticket Structure

Story Types

- **Stories:** Individual units of work (1–3 points ideally)
- **Sub-tasks:** For breaking down work within a story (optional)
- **Epics:** Ties together units of work into cohesive solution, used by PMs to tie to initiatives and business objectives

Epic Usage

- Primarily owned by Product Management
- Used for grouping related work that maps to business initiatives
- Contains CapEx/OpEx designation for financial tracking
- Engineering can propose epics but should coordinate with PM

Epic, Story, or Task?

When deciding how to structure work:

- **Use sub-tasks when:** Work is tightly coupled and should ship together
- **Use separate stories when:** Work can be independently delivered/tested
- **Use an epic when:** Multiple stories contribute to a larger business objective

Best Practices

Writing Stories

- Clear acceptance criteria.
- It's worth repeating: **CLEAR ACCEPTANCE CRITERIA.**
 - AC Razor: can someone else, even from another team or non-technical, read the ticket and understand what DONE looks like?
- Link to relevant artifacts (tech specs, RFC, other PRs, Slack convos, etc.)
- This doesn't have to be burdensome. You don't have to be a **Jack Danger** level writer to write a great story. You do have to have sufficient information that communicates what's being solved (Epic or Problem Statement), what's being done (Issue Title or a Statement of Work) how we will know it's in fact solved (Acceptance Criteria), and what we thought it'd cost us to do so (Story Points).

PR Mapping

- No strict 1:1 story-to-PR requirement

- Multiple small PRs is encouraged, provided they encapsulate a meaningfully bounded unit of work and value
 - Shortens feedback cycle, lowers time required by peers

Examples

Good Story Sizing

- ✓ "Upgrade eipservices Java dependencies" – 2 points
- ✓ "Add React component for attestation workflow" – 3 points
- ✓ "Implement new logging format for Stork" – 1 point

Needs Breakdown

- ✗ "Modernize request intake system" – Too vague, needs epic + stories
- ✗ "Fix all security vulnerabilities in platformservices" – Break out into pointable, specific upgrades

Workflow States

- **To Do:** Refined and ready for pickup
- **In Progress:** Actively being worked
- **Review**
 - **Code Review:** PR submitted, feedback cycle commenced
 - **QA/UAT:** Gathering sufficient evidence with appropriate stakeholders/owners before releasing to users
- **Done:** Merged and deployed

Last updated: August 2025